

Explicação dos Exercícios

exam_rank04

[Explicação dos Exercícios — exam_rank04](#)

[LEVEL 1](#)

[LEVEL 2](#)

[LEVEL 3](#)

[Resumo das técnicas usadas em cada exercício](#)

Explicação dos Exercícios — exam_rank04

Este ficheiro explica, em detalhe, a lógica de cada um dos 7 exercícios do exame (Level 1, 2 e 3). Para cada exercício: o que é pedido, a estratégia usada, uma explicação linha-a-linha da lógica, e porque é que a solução funciona (incluindo casos-limite).

LEVEL 1

1. `array_rotation_detector` — Detecção de rotação de arrays

Enunciado: dados dois arrays, dizer se `arr2` é uma rotação (à esquerda ou à direita) de `arr1`.

Ideia central: uma rotação de um array nada mais é do que “cortar” o array numa posição e colar o pedaço do início atrás do pedaço do fim (ou vice-versa). Em vez de tentar perceber matematicamente qual é o deslocamento, a solução testa *todas* as rotações possíveis de `arr1` e vê se alguma delas é igual a `arr2`.

Função auxiliar `twister(nums, n)`:

```
def twister(nums, n) -> list:
    if len(nums) == 0:
        return []
    n = n % len(nums)
    return nums[-n:] + nums[:-n]
```

- Roda a lista `nums` para a direita `n` posições.
- `nums[-n:]` são os últimos `n` elementos (o “fim” da lista).
- `nums[:-n]` são todos os elementos exceto os últimos `n` (o “resto do início”).
- Ao colar `fim + início`, obtém-se a lista rodada. Exemplo: `twister([1,2,3,4,5], 2) → [4,5] + [1,2,3] = [4,5,1,2,3]`.
- `n = n % len(nums)` evita erros quando `n` é maior que o tamanho da lista (roda “à volta”).
- Caso especial: lista vazia devolve `[]` logo de início (evita divisão por zero no `%`).

Função principal:

```
def array_rotation_detector(arr1, arr2) -> bool:
    if len(arr1) == 0:
        if len(arr2) == 0:
            return True
    if len(arr1) != len(arr2):
        return False
    i = 0
    while i < len(arr1):
        if twister(arr1, i) == arr2:
            return True
        i += 1
    return False
```

- **Caso base:** se ambos os arrays estão vazios, são considerados rotações um do outro → `True`.
- **Comprimentos diferentes:** nunca podem ser rotações → `False` imediato.
- **Ciclo principal:** percorre todos os deslocamentos possíveis `i` de `0` até `len(arr1)-1` e, para cada um, roda `arr1` por `i` posições com `twister`. Se em algum momento a rotação for igual a `arr2`, confirma-se a rotação e devolve `True`.
- Se nenhuma rotação coincidir, devolve `False`.

Porquê funciona: existem exatamente `len(arr1)` rotações distintas possíveis (para `n` a variar de `0` a `len-1`), por isso testar todas garante que nenhuma hipótese é ignorada. É uma solução por *força bruta*, com complexidade $O(n^2)$ (para cada `i`, cria-se uma nova lista e compara-se), mas simples e correta.

2. `constellation_mapper` — Mapeamento de estrelas numa grelha

Enunciado: a partir de uma lista de coordenadas (`linha, coluna`), construir uma grelha `size x size` com `“*”` nas posições indicadas e `“.”` nas restantes, ignorando coordenadas fora dos limites e duplicadas.

```
def constellation_mapper(stars, size) -> list[str]:
    grid = []
    i = 0
    while i < size:
        grid.append(["."] * size)
```

```

        i += 1
    for row, col in stars:
        if 0 <= row < size and 0 <= col < size:
            grid[row][col] = "*"
    result = []
    for row in grid:
        result.append("".join(row))
    return result

```

- **Passo 1 — criar a grelha vazia:** um ciclo `while` cria `size` linhas, cada uma sendo uma lista de `size` pontos (`"."`). Usa-se lista de listas (matriz) em vez de strings diretamente, porque strings em Python são *imutáveis* — não se pode fazer `grid[row][col] = "*"` diretamente numa string.
- **Passo 2 — marcar as estrelas:** percorre-se cada par `(row, col)` em `stars`. A condição `0 <= row < size and 0 <= col < size` garante que só se marcam coordenadas dentro dos limites da grelha (ignora automaticamente as que estão fora).
- **Duplicados são tratados “de graça”:** marcar a mesma posição duas vezes com `"*"` não tem efeito extra — a posição fica simplesmente com `"*"`, exatamente como pedido no enunciado.
- **Passo 3 — converter para strings:** cada linha (lista de caracteres) é transformada numa string com `"".join(row)`, porque o resultado final deve ser `list[str]` e não `list[list[str]]`.

Porquê funciona: a matriz auxiliar permite alterações eficientes célula a célula; a verificação de limites filtra coordenadas inválidas; a conversão final respeita o tipo de retorno pedido.

3. `list_intersection_finder` — Interseção de várias listas

Enunciado: encontrar os elementos comuns a *todas* as listas dadas, sem duplicados e ordenados.

```

def list_intersection_finder(lists) -> list[int]:
    i = 0
    if not lists:
        return []
    while i < len(lists):
        if lists[i] == []:
            return []
        i += 1
    if len(lists) == 1:
        return lists[0]
    result = []
    for value in lists[0]:
        if all(value in lista for lista in lists):
            if value not in result:
                result.append(value)
    return result

```

- **Verificação 1:** se `lists` (a lista externa) estiver vazia → devolve `[]` (não há nenhuma lista para interseção).
- **Verificação 2:** percorre todas as listas internas; se alguma estiver vazia, a interseção com um conjunto vazio é sempre vazia → devolve `[]` imediatamente (otimização, evita trabalho desnecessário).
- **Caso especial de 1 lista só:** se só existe uma lista, a “interseção” é ela própria (sem alterações).
- **Lógica principal:** percorre-se apenas os valores da **primeira** lista (`lists[0]`) — não é preciso testar valores de outras listas, porque um valor só pode estar na interseção se também estiver na primeira. Para cada `value`, usa-se `all(value in lista for lista in lists)`, que verifica se esse valor existe em **todas** as listas (incluindo a primeira, o que é redundante mas inofensivo).
- **Remoção de duplicados:** antes de adicionar `value` ao `result`, verifica-se `if value not in result`, para garantir que cada valor aparece só uma vez no resultado final.
- **Ordenação:** como se percorre `lists[0]` pela ordem em que já vem (e as listas de entrada estão ordenadas, segundo o enunciado), o resultado sai naturalmente ordenado.

Nota de eficiência: `value in lista` é $O(n)$ por lista, e isto corre dentro de outro ciclo — a complexidade é $O(n^2)$ no pior caso. Para os tamanhos deste exercício é perfeitamente aceitável, mas poderia ser otimizado com `set` para grandes volumes de dados.

LEVEL 2

4. `sliding_window_maximum` — Máximo em janela deslizante

Enunciado: dado um array e um tamanho de janela `k`, calcular o valor máximo dentro de cada posição da janela à medida que ela desliza da esquerda para a direita.

```
def sliding_window_maximum(nums, k) -> list[int]:
    if not nums or k <= 0 or len(nums) < k:
        return []
    i = 0
    result = []
    while i < len(nums) - (k - 1):
        j = 0
        temp = i
        biggest = nums[i]
        while j < k:
            if nums[temp] > biggest:
                biggest = nums[temp]
            j = j + 1
            temp += 1
        result.append(biggest)
        i += 1
    return result
```

- **Verificações iniciais:** lista vazia, `k` inválido (`<= 0`) ou `k` maior que o tamanho da lista → não há janelas possíveis, devolve `[]`.
- **Ciclo externo (`i`):** representa a posição inicial de cada janela. O número total de janelas é `len(nums) - k + 1`, por isso o ciclo corre enquanto `i < len(nums) - (k - 1)` (equivalente a `i <= len(nums) - k`).
- **Ciclo interno (`j` / `temp`):** para cada janela, percorre-se os `k` elementos a partir da posição `i` (`temp` vai de `i` até `i + k - 1`) e guarda-se o maior valor encontrado (`biggest`).
- Cada máximo encontrado é adicionado a `result`, e passa-se à janela seguinte incrementando `i`.

Porquê funciona: é uma simulação direta e literal da definição do problema — “para cada posição da janela, encontra o máximo” — daí a chamada de solução por *força bruta*. Complexidade $O(n \cdot k)$, que é aceitável para este nível de exercício (uma solução mais avançada usaria uma *deque* para $O(n)$, mas não é exigido aqui).

5. `merge_sorted_list` — Juntar várias listas ordenadas

Enunciado: juntar várias sublistas (já ordenadas individualmente) numa só lista final, ordenada, mantendo duplicados e ignorando sublistas vazias.

```
def merge_sorted_list(lists) -> list[int]:
    if not lists:
        return []
    i = 0
    result = []
    while i < len(lists):
        result += sorted(lists[i])
        i += 1
    return sorted(result)
```

- **Verificação inicial:** se a lista externa estiver vazia, não há nada a juntar → `[]`.
- **Ciclo de junção:** percorre-se cada sublista e vai-se acrescentando (`+=`) o seu conteúdo (já ordenado outra vez com `sorted`, embora o enunciado garanta que já vêm ordenadas — é uma “rede de segurança”) ao resultado.
- **Sublistas vazias:** `sorted([])` devolve `[]`, e concatenar `[]` com `result` não tem efeito — por isso as sublistas vazias são automaticamente “ignoradas” sem código extra.
- **Ordenação final:** depois de juntar tudo, ordena-se o `result` completo com `sorted(result)`, garantindo que a lista final está corretamente ordenada de ponta a ponta (não bastaria apenas concatenar, porque a ordem entre sublistas diferentes não estaria garantida).

Nota: esta não é a implementação “clássica” de *merge* de k-listas ordenadas (que normalmente usa uma *heap* ou junção por comparação em $O(n \log k)$ para aproveitar o facto de já estarem ordenadas); aqui opta-se por juntar tudo e ordenar de novo ($O(n \log n)$), o que é mais simples e perfeitamente válido para o que é pedido.

Nota sobre o enunciado: o exemplo do docstring contém um erro de escrita (`[1, , 5]` — uma vírgula a mais), mas isso é apenas texto de comentário; o código foi testado com `[1, 4, 5]` e funciona corretamente. A assinatura no docstring (`-> [list[int]:]`) também tem um pequeno erro de sintaxe, mas de novo é só o texto do enunciado — a implementação real usa a assinatura correta.

6. `palindrome_partitioner` — Número mínimo de cortes para obter só palíndromos

Enunciado: dada uma string, calcular o número mínimo de cortes necessários para que cada pedaço resultante seja um palíndromo.

```
def palindrome_partitioner(s):
    n = len(s)
    if n <= 1:
        return 0
    dp = list(range(n))
    for i in range(n):
        for l, r in [(i, i), (i, i + 1)]:
            while l >= 0 and r < n and s[l] == s[r]:
                if l == 0:
                    dp[r] = 0
                else:
                    dp[r] = min(dp[r], dp[l - 1] + 1)
                l -= 1
                r += 1
    return dp[-1]
```

Este é o exercício mais complexo — usa **programação dinâmica (DP)** combinada com **expansão a partir do centro** (a técnica clássica para detetar palíndromos).

- **Ideia de dp:** `dp[r]` guarda o número mínimo de cortes necessários para que a substring `s[0..r]` (do início até à posição `r`, inclusive) seja toda composta por palíndromos.
- **Inicialização `dp = list(range(n))`:** no pior caso, cada caractere é o seu próprio palíndromo, por isso a solução “pior possível” para `dp[r]` é `r` cortes (um corte entre cada par de caracteres). Começa-se com esse valor “pessimista” e vai-se melhorando (reduzindo) à medida que se encontram palíndromos maiores.
- **Expansão a partir do centro:** para cada índice `i`, testam-se dois tipos de centro:
 - `(i, i)` — centro num único caractere (palíndromos de comprimento ímpar, ex: “aba”).
 - `(i, i+1)` — centro entre dois caracteres (palíndromos de comprimento par, ex: “abba”).
 - O `while` expande `l` (esquerda) e `r` (direita) para fora, enquanto os caracteres em `s[l]` e `s[r]` forem iguais — ou seja, enquanto a substring `s[l..r]` continuar a ser um palíndromo.
- **Atualização de dp:** sempre que se confirma que `s[l..r]` é um palíndromo:
 - Se `l == 0`, isso significa que a substring palíndroma vai desde o início da string até `r` — logo, **zero cortes** são necessários para essa parte (`dp[r] = 0`).
 - Caso contrário, o melhor número de cortes até `r` é o melhor número de cortes até `l-1` (a parte antes do palíndromo atual) **mais um corte** (o corte que separa essa parte do palíndromo atual). Usa-se `min(...)` porque pode já existir uma solução melhor encontrada por outro centro de expansão.
- **Resultado final:** `dp[-1]`, ou seja, `dp[n-1]`, o mínimo de cortes necessário para toda a string.

Exemplo mental com “aab”: - `dp` começa como `[0, 1, 2]`. - Expansão em `i=0`: centro `(0,0)` → “a” é palíndromo, `l==0` → `dp[0]=0`. Centro `(0,1)`: `s[0]='a'`, `s[1]='a'`, são iguais → `l==0` → `dp[1]=0`. - Expansão em `i=1`: centro `(1,1)` → “a” é palíndromo, `l=1` não é 0 → `dp[1] = min(dp[1], dp[0]+1) = min(0, 1) = 0`. - Expansão em `i=2`: centro `(2,2)` → “b” é palíndromo, `l=2` → `dp[2] = min(dp[2], dp[1]+1) = min(2, 1) = 1`. - Resultado: `dp[2] = 1` → **1 corte**, que corresponde a “aa” | “b”. ✓

Porquê funciona: ao combinar todos os palíndromos possíveis (centrados em cada posição) com a relação `dp[r] = min(dp[r], dp[l-1] + 1)`, garante-se que se explora sempre a melhor combinação de cortes

possível, sem nunca perder a hipótese ótima.

LEVEL 3

7. `package_dependency_resolver` — Ordenação topológica (algoritmo de Kahn)

Enunciado: dado um dicionário de pacotes e as suas dependências, determinar uma ordem válida de instalação (dependências primeiro), devolvendo `[]` se houver ciclos, e desempatando alfabeticamente quando há escolha.

```
def package_dependency_resolver(packages) -> list[str]:
    if not packages:
        return []
    result = []
    indegree = {}
    graph = {}
    for package in packages:
        indegree[package] = 0
        graph[package] = []
    for package in packages:
        for dep in packages[package]:
            if dep in packages:
                graph[dep].append(package)
                indegree[package] += 1
    queue = []
    for package in indegree:
        if indegree[package] == 0:
            queue.append(package)
    queue.sort()
    while queue:
        current = queue.pop(0)
        result.append(current)
        for nxt in graph[current]:
            indegree[nxt] -= 1
            if indegree[nxt] == 0:
                queue.append(nxt)
        queue.sort()
    if len(result) != len(packages):
        return []
    return result
```

Este exercício implementa o **algoritmo de Kahn** para ordenação topológica de um grafo dirigido.

- **Conceitos-chave:**

- `graph[X]` = lista de pacotes que **dependem de** `X` (ou seja, arestas de “`X` → quem precisa de `X`”).
- `indegree[X]` = número de dependências que `X` ainda tem por resolver (quantas “setas” apontam para `X`).

- **Passo 1 — inicializar as estruturas:** cada pacote começa com `indegree = 0` e uma lista vazia em `graph`.
- **Passo 2 — construir o grafo:** para cada pacote e cada uma das suas dependências `dep`, se `dep` for um pacote válido (está nas chaves do dicionário — isto implementa a regra “ignorar dependências inexistentes”), adiciona-se uma aresta `dep → package` em `graph`, e incrementa-se o `indegree` do `package` (porque agora tem mais uma dependência pendente).
- **Passo 3 — fila inicial:** todos os pacotes com `indegree == 0` (sem dependências, ou cujas dependências não existem no dicionário) entram na fila `queue`. A fila é ordenada alfabeticamente (`queue.sort()`) para garantir a regra de **desempate determinístico**.
- **Passo 4 — processamento (o coração do algoritmo de Kahn):**
 - Retira-se o primeiro elemento da fila (`queue.pop(0)`) — como a fila está sempre ordenada, isto garante que se processa sempre o pacote alfabeticamente mais pequeno entre os disponíveis.
 - Esse pacote é adicionado ao `result` (já pode ser “instalado”, pois todas as suas dependências já foram processadas).
 - Para cada pacote `nxt` que dependia de `current` (`graph[current]`), decrementa-se o seu `indegree`

(uma dependência a menos por resolver). Se o `indegree` chegar a `0`, esse pacote já pode entrar na fila.

- No fim de cada iteração, a fila é reordenada (`queue.sort()`), para manter sempre a prioridade alfabética entre os pacotes disponíveis no momento.
- **Deteção de ciclos:** se existir um ciclo (ex: `X` depende de `Y` e `Y` depende de `X`), nenhum dos dois nunca chega a `indegree == 0`, por isso nunca entram na fila e nunca são adicionados a `result`. No final, compara-se `len(result)` com `len(packages)`: se forem diferentes, significa que sobraram pacotes por processar (porque estão presos num ciclo) → devolve-se `[]`.

Exemplo mental com `{"web": [], "api": [], "frontend": ["web"], "backend": ["api"]}`: - `indegree`: `web=0, api=0, frontend=1, backend=1`. - Fila inicial (ordenada): `["api", "web"]`. - Processa `"api"` → `result=["api"]`. `backend` perde uma dependência → `indegree[backend]=0` → entra na fila. Fila agora: `["backend", "web"]` (reordenada). - Processa `"backend"` → `result=["api", "backend"]`. - Processa `"web"` → `result=["api", "backend", "web"]`. `frontend` perde uma dependência → `indegree[frontend]=0` → entra na fila. - Processa `"frontend"` → `result=["api", "backend", "web", "frontend"]`. ✓ Corresponde exatamente ao exemplo do enunciado.

Porquê funciona: o algoritmo de Kahn garante que um pacote só é processado depois de **todas** as suas dependências terem sido processadas (porque só entra na fila quando `indegree` chega a zero), e a reordenação constante da fila garante a regra de desempate alfabético exigida pelo enunciado. A verificação final do tamanho do `result` é a forma clássica de detetar ciclos neste algoritmo.

Resumo das técnicas usadas em cada exercício

Exercício	Técnica principal
array_rotation_detector	Força bruta — testar todas as rotações possíveis
constellation_mapper	Construção de matriz + validação de limites
list_intersection_finder	Verificação com <code>all()</code> sobre a primeira lista como referência
sliding_window_maximum	Simulação direta da janela deslizante (força bruta)
merge_sorted_list	Concatenação + ordenação final
palindrome_partitioner	Programação dinâmica + expansão a partir do centro
package_dependency_resolver	Ordenação topológica — Algoritmo de Kahn (BFS com filas de indegree)

Todos os scripts foram executados e os resultados obtidos coincidem com os exemplos apresentados nos enunciados de cada exercício.